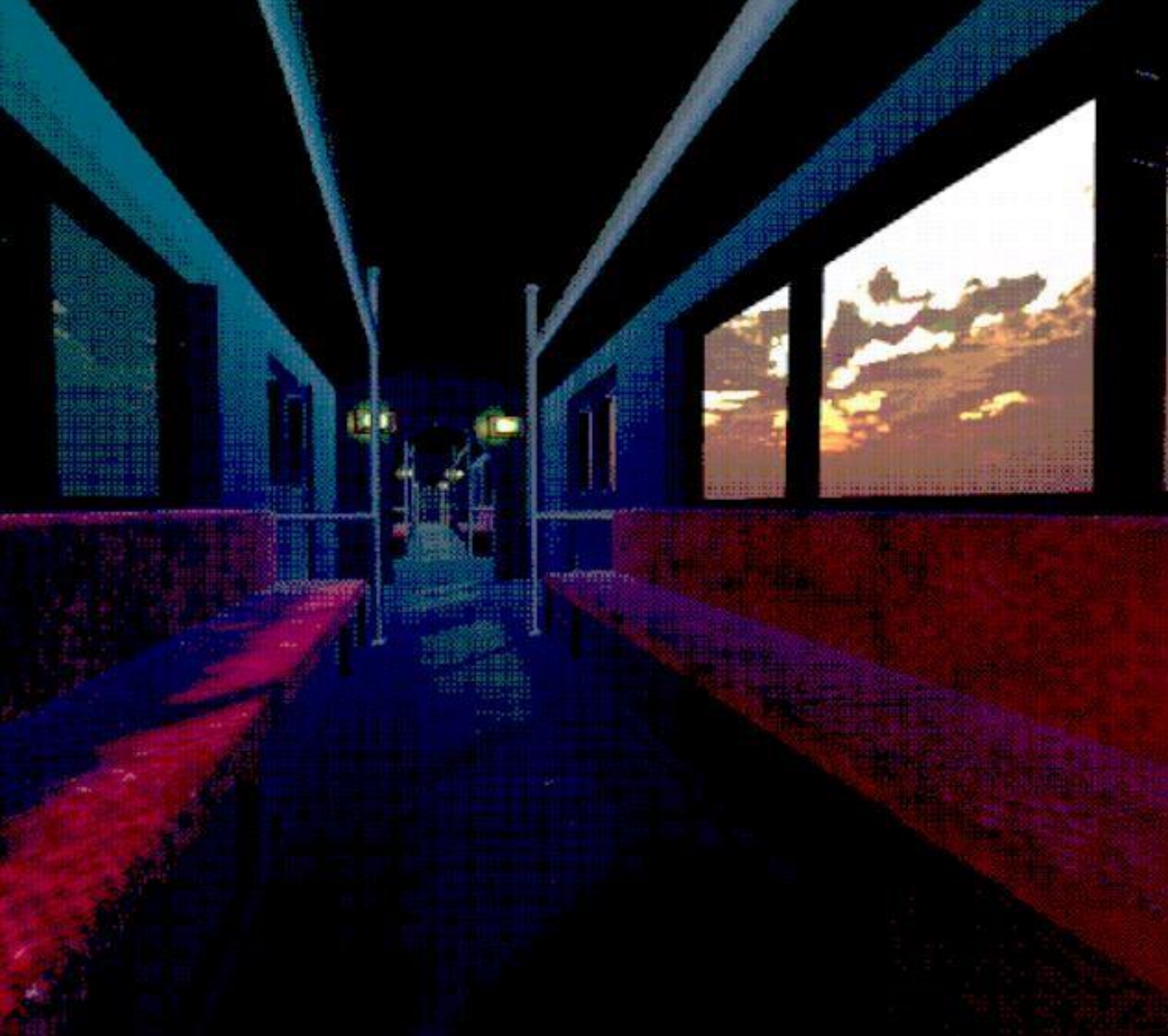


Manual técnico para Estaciones



Alumno: Silva Quintero Ricardo Santiago
Número de cuenta: 319012220
Fecha de entrega: 20/05/2026
Materia: Temas selectos de ingeniería en
computación 3 AR y VR 2026-2

Contenido

1. Descripción General del Sistema	4
2. Entorno de Desarrollo	4
3. Cronograma de actividades	4
4. Análisis de Tecnología con Justificación de Uso	6
Unity 6000.3.7f1.....	6
Universal Render Pipeline (URP)	7
Shader Graph (URP)	7
Blender	7
FL Studio	8
OpenXR + Meta XR SDK.....	8
GitHub	¡Error! Marcador no definido.
5. Estructura del Proyecto	8
5.1 Organización de carpetas	8
5.2 Escenas del proyecto	9
6. Scripts — Descripción Técnica	9
PlayerController.cs.....	9
Inter.cs	10
InfiniteTrain.cs	10
StopTrain.cs.....	11
Teleport.cs	11
Portal.cs.....	12
LevelManager.cs.....	12
SaveSystem.cs	12
MirarJugador.cs	13
OjoMironGrande.cs.....	13
7. Sistema de Renderizado y Shader.....	14
7.1 Configuración del Universal Render Pipeline	14
7.2 Shader de filtro pixelado	14
8. Iluminación.....	15
8.1 Mapa1 — Vecindario Tranquilo: iluminación dinámica.....	15
8.2 Mapa2 — Baños: iluminación bakeada	16
9. Audio	17
10. Sistema de Partículas VFX	18

11. Generación de Builds	18
11.1 Build para PC — Windows.....	18
11.2 Build para Meta Quest 2 — APK.....	18
12. Repositorio y Control de Versiones	19
13. Consideraciones Técnicas y Limitaciones Conocidas.....	19

1. Descripción General del Sistema

Estaciones es un videojuego de exploración en 3D desarrollado en Unity 6000.3.7f1 con Universal Render Pipeline. El proyecto genera dos builds independientes desde el mismo código fuente: una aplicación ejecutable para PC en Windows y un paquete APK para Meta Quest 2 mediante OpenXR. Ambas builds comparten la totalidad de la lógica de juego, los assets y las escenas; la diferencia entre ellas reside en la configuración del renderer, los ajustes de calidad y el plugin de XR activo.

El juego está estructurado en torno a cinco escenas jugables que representan los espacios de la experiencia: el menú principal, la escena intermedia de transición, el Tren como hub central, y dos niveles de exploración denominados Mapa0 y Mapa1. Existe además una sexta escena, MapaPruebas, utilizada exclusivamente durante el desarrollo para probar mecánicas y assets de forma aislada sin afectar las escenas de producción.

2. Entorno de Desarrollo

El proyecto fue desarrollado íntegramente con las siguientes herramientas, versiones y configuraciones. Unity 6000.3.7f1 actúa como motor principal y entorno de integración. El pipeline de renderizado es Universal Render Pipeline en su versión incluida con Unity 6, configurado con dos Renderer Assets distintos: pc_renderer para la build de escritorio y mobile_renderer para la build de Meta Quest. Blender fue utilizado para el modelado, UV mapping, rigging y exportación de todos los assets 3D del proyecto en formato .fbx. FL Studio se empleó para la producción y exportación del audio ambiental original de cada escena. El control de versiones se gestiona con Git y GitHub, con Git LFS habilitado para archivos binarios pesados como modelos, texturas y audio.

Para la build de Meta Quest, el proyecto requiere adicionalmente el paquete XR Plugin Management con el proveedor OpenXR activo, el paquete Meta XR Core SDK y el Android Build Support instalado en el editor de Unity con el Android SDK y JDK correspondientes. La firma del APK se realiza mediante un keystore propio del equipo generado durante la configuración inicial del proyecto para plataforma Android.

3. Cronograma de actividades

1. Inicio del proyecto y Planificación	11/02	24/02	14
› Definir idea central y propuesta del juego	11/02	12/02	2
› Establecer objetivos, alcance y plataforma	12/02	13/02	2
› Investigar referentes y juegos similares	13/02	15/02	3

› Seleccionar motor y herramientas de desarrollo	15/02	16/02	2
› Configurar repositorio Git y estructura base	18/02	20/02	3
› Definir canales de comunicación	20/02	22/02	3
› Definir niveles principales y flujo de juego	20/02	24/02	5
› Primer borrador del GDD	22/02	24/02	3
› Cerrar y aprobar el GDD con el equipo	22/02	24/02	3
› Configurar entorno de desarrollo del proyecto	21/02	24/02	4
2. Construcción jugable básica	25/02	24/03	28
› Prototipo de movimiento y control del personaje	25/02	28/02	4
› Sistema de físicas y detección de colisiones	28/02	04/03	5
› Prototipado de niveles base	04/03	08/03	5
› Sistema de cámara y seguimiento del personaje	08/03	11/03	4
› Game loop principal: inicio, pausa, fin	11/03	17/03	7
› UI básica: menú principal en partida	14/03	17/03	4
› Sistema de audio base prototipado	17/03	20/03	4
› Sistema de guardado y carga de partidas	20/03	22/03	3
› Pruebas internas de jugabilidad y ajustes	22/03	24/03	3
3. Construcción visual/técnica	25/03	02/05	39
› Modelado 3D de escenarios y entornos;	25/03	04/04	11
› Efectos visuales (VFX): partículas y efectos	04/04	11/04	8
› Iluminación de escenarios	11/04	17/04	7
› Postprocesado y shaders base	17/04	27/04	11
› Modelado 3D assets: personajes, enemigos y props	21/04	28/04	8
› Sistema de animaciones de personaje/enemigos	25/04	02/05	8
› UI final e integración de faltantes anteriores	25/04	02/05	8
4. Optimización y pruebas	03/05	12/05	10
› Sesiones de QA con usuarios externos e integración de cambios	03/05	08/05	6

› Balancear dificultad según métricas de prueba	08/05	10/05	3
› Corrección de bugs identificados e integracion de faltantes anteriores	10/05	12/05	3
5. Documentación y cierre	13/05	21/05	9
› Documentación técnica del proyecto	13/05	15/05	3
› Manual de usuario y guía de jugabilidad	15/05	17/05	3
› Actualizar y finalizar el GDD	17/05	18/05	2
› Generar build final para la plataforma	18/05	19/05	2
6. Preparación presentación	22/05	25/05	4
› Elaborar guión y estructura de la presentación	22/05	23/05	2
› Diseñar slides con identidad visual del juego	23/05	24/05	2
› Preparar demo jugable para exposición	24/05	25/05	2
› Ensayar presentación completa	24/05	25/05	2
7. Presentación final	26/05	26/05	1
› Exposición del proyecto	26/05	26/05	1

4. Análisis de Tecnología con Justificación de Uso

A continuación, se presenta el análisis técnico de cada herramienta y tecnología utilizada en el proyecto, con la justificación de su elección dentro del contexto específico de Estaciones:

Unity 6000.3.7f1

Unity es el motor principal del proyecto. Se eligió la versión 6000.3.7f1, conocida como Unity 6 LTS, por ser la versión más estable con soporte extendido disponible al inicio del semestre. Esta versión incluye mejoras sustanciales en el rendimiento del Universal Render Pipeline y compatibilidad nativa con OpenXR para Meta Quest. Unity permite gestionar en un mismo entorno la lógica de juego, el sistema de animación, el Timeline para la sincronización del tren, los sistemas de partículas VFX y las builds para múltiples plataformas desde un solo proyecto. Su soporte oficial de URP con Shader Graph fue determinante para la implementación del shader pixelado y el bake de iluminación integrado mediante el Lightmapper progresivo fue fundamental para el Nivel 2.

Universal Render Pipeline (URP)

El Universal Render Pipeline fue elegido sobre el Built-in Pipeline y el HDRP por ser el único que ofrece un balance óptimo entre calidad visual y rendimiento en hardware limitado. El Built-in Pipeline no soporta Shader Graph de forma nativa ni tiene un sistema de postprocesado modular. El HDRP, aunque visualmente superior, es exclusivo de hardware de alto rendimiento y no es compatible con Meta Quest.

El URP aporta al proyecto varias ventajas concretas. Sus Renderer Features personalizadas sirven como punto de extensión donde se aplica el filtro pixelado como un Full Screen Render Pass sobre la imagen final. Además, su compatibilidad multiplataforma permite que el mismo proyecto genere una build de PC y una build de Meta Quest ambas usando el mismo pipeline.

Shader Graph (URP)

Shader Graph es la herramienta visual de creación de shaders integrada en Unity con URP. Se usó para desarrollar el shader de filtro pixelado que se aplica permanentemente sobre la vista del jugador. Técnicamente, este shader funciona como un Full Screen Shader que recibe la textura de la cámara renderizada, divide las coordenadas UV de la pantalla entre un valor de resolución de cuadrícula configurable, las redondea con la función Floor y las multiplica de nuevo, generando bloques de píxeles uniformes que simulan la resolución reducida de pantallas retro. El parámetro de tamaño de píxel es ajustable en tiempo de ejecución, lo que permite modular la intensidad del efecto por nivel en iteraciones futuras.

Shader agua, por otro lado, se creó con la finalidad de brindar un acercamiento más realista al nivel 2, debido a que los poolrooms (Inspiración para la creación del nivel 2) se componen principalmente de paredes de azulejos, cuerpos de agua y charcos.

Finalmente, los shaders ocultador y objeto oculto fueron utilizados para Stencil dentro del Nivel 2, creando geometría imposible, concordando con el surrealismo del videojuego en general.

Blender

Blender es el software de modelado 3D utilizado para crear todos los assets del juego: escenarios, personajes y elementos decorativos. Se eligió por ser gratuito, ampliamente compatible con Unity mediante exportación en formato .fbx y suficientemente potente para el nivel de detalle low-poly requerido. En Estaciones se usó para modelar todos los elementos de los niveles con una reducción deliberada de polígonos por objeto como estrategia de optimización, para el UV mapping y preparación de texturas orientadas al bake de iluminación en Unity.

La restricción del proyecto que prohíbe el sistema de terrenos nativo de Unity se resuelve directamente con Blender, que permite construir geometría de suelo personalizada con mallas controladas y sin dependencia de herramientas del motor.

FL Studio

FL Studio es el DAW utilizado para producir el audio ambiental original del proyecto. Se eligió sobre alternativas gratuitas porque permite crear composiciones en capas con mayor control sobre la atmósfera sonora, lo cual es fundamental en un juego de exploración donde el audio es parte central de la experiencia. Se produjeron tres piezas de audio ambiental: el sonido del viento, vibración de rieles y desaceleración del Tren, la música del Vecindario Tranquilo y la música de Baños. Todos los archivos se exportaron en formato .mp3 y se importaron a Unity.

OpenXR + Meta XR SDK

OpenXR es el estándar abierto de la industria para el desarrollo de aplicaciones de realidad virtual, respaldado por el Khronos Group. Se eligió sobre el SDK propietario de Meta porque desacopla el proyecto de una plataforma específica: el mismo proyecto puede ejecutarse en Meta Quest 2, Quest 3 y cualquier dispositivo compatible con OpenXR sin modificaciones de código. La configuración técnica incluye Unity XR Plugin Management con OpenXR como proveedor XR activo, la Meta OpenXR Feature habilitada para compatibilidad con las extensiones del Quest, el Android Build Target con Gradle configurado para generar un APK firmado compatible con el Meta Quest Developer Hub, Single Pass Instanced Rendering activo para el renderizado estéreo eficiente, y Fixed Foveated Rendering en nivel medio para reducir la carga de shaders en la periferia del campo visual donde el ojo no distingue detalle.

5. Estructura del Proyecto

5.1 Organización de carpetas

El proyecto sigue la estructura de carpetas estándar que genera Unity dentro del directorio Assets. Cada tipo de recurso tiene su propia carpeta dedicada para mantener el proyecto organizado y facilitar la navegación. La estructura principal es la siguiente.

Assets/	
├─ 3D/	Modelos importados desde Blender en formato .fbx
├─ Audios/	Clips de audio ambiental exportados desde FLStudio
├─ Escenas/	Las seis escenas del proyecto (.unity)
├─ Materials/	Materiales y configuraciones de shader
├─ Prefabs/	Objetos prefabricados reutilizables
├─ Scripts/	Todos los scripts en C#
├─ Settings/	Renderer Assets de URP y configuración de XR
└─ Textures/	Mapas de textura y lightmaps bakeados

5.2 Escenas del proyecto

El proyecto contiene seis escenas organizadas en la carpeta Escenas. MainMenu es la escena de inicio donde el jugador accede al menú principal e interactúa con el letrero de opciones. BetweenPlayAndTrain es la escena intermedia que muestra la ventana estilo Windows 7 durante aproximadamente 30 segundos antes de cargar el Tren. Mapa0 es el Tren, hub central del juego desde donde el jugador viaja entre mundos. Mapa1 corresponde al primer nivel jugable, el Vecindario Tranquilo. Mapa2 corresponde al segundo nivel jugable, los Baños. MapaPruebas es una escena auxiliar de desarrollo, no incluida en el flujo de juego de producción, utilizada para pruebas de assets y mecánicas de forma aislada.



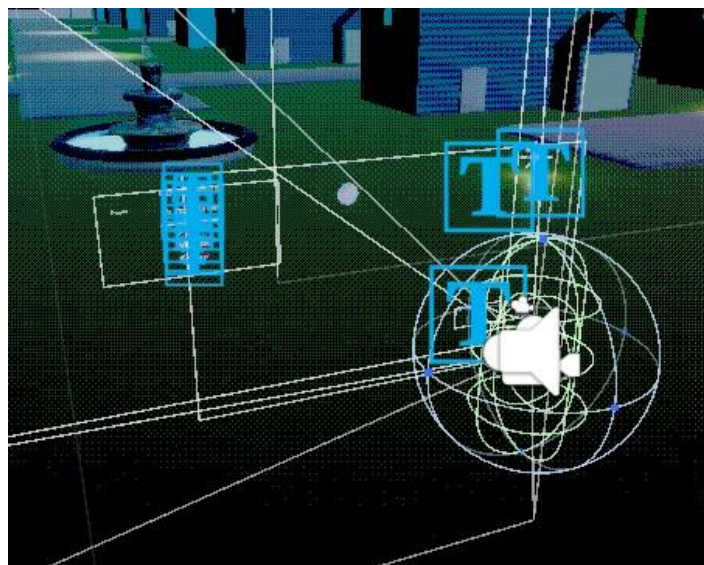
(Figura 1. Contenido de escenas)

6. Scripts — Descripción Técnica

Todos los scripts del proyecto están escritos en C# y se encuentran en la carpeta Assets/Scripts. A continuación, se describe la responsabilidad de cada uno, su ubicación dentro de la jerarquía de escenas y las dependencias relevantes entre ellos.

PlayerController.cs

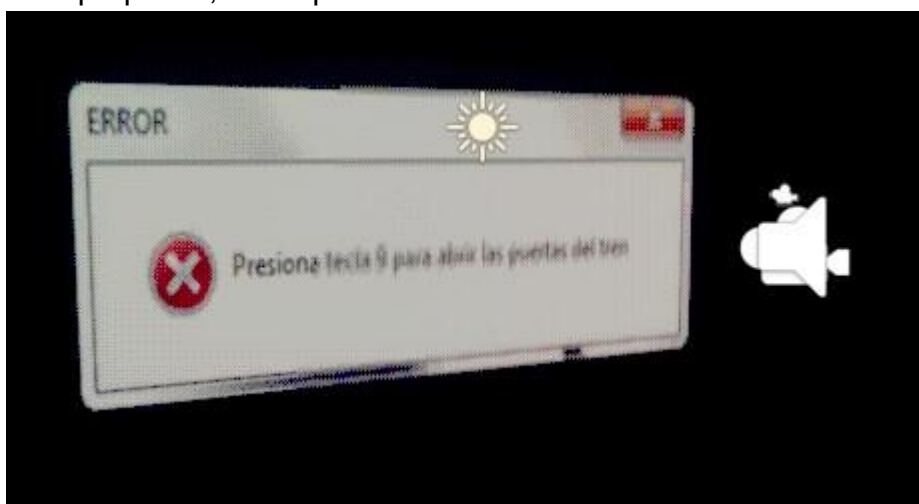
Es el script central del personaje jugable. Se encarga de leer las entradas del teclado y el ratón para trasladar y rotar al personaje en el espacio 3D. Gestiona el movimiento en las cuatro direcciones mediante las teclas WASD, la velocidad de carrera al mantener Shift izquierdo y la orientación de la cámara según el movimiento del ratón. Este script está presente en todas las escenas jugables y se adjunta al GameObject del jugador. No depende de otros scripts del proyecto, pero es referenciado por StopTrain y Teleport para controlar el estado del jugador durante las transiciones.



(Figura 2. Player dentro de la escena)

Inter.cs

Controla el comportamiento de la escena BetweenPlayAndTrain. Al iniciarse la escena, activa la ventana de estilo Windows 7 con el mensaje de instrucciones y lanza una corrutina con un temporizador de 30 segundos. Al cumplirse el tiempo, llama al SceneManager de Unity para cargar la escena Mapa0, que corresponde al Tren. Es un script de un solo propósito, sin dependencias externas.

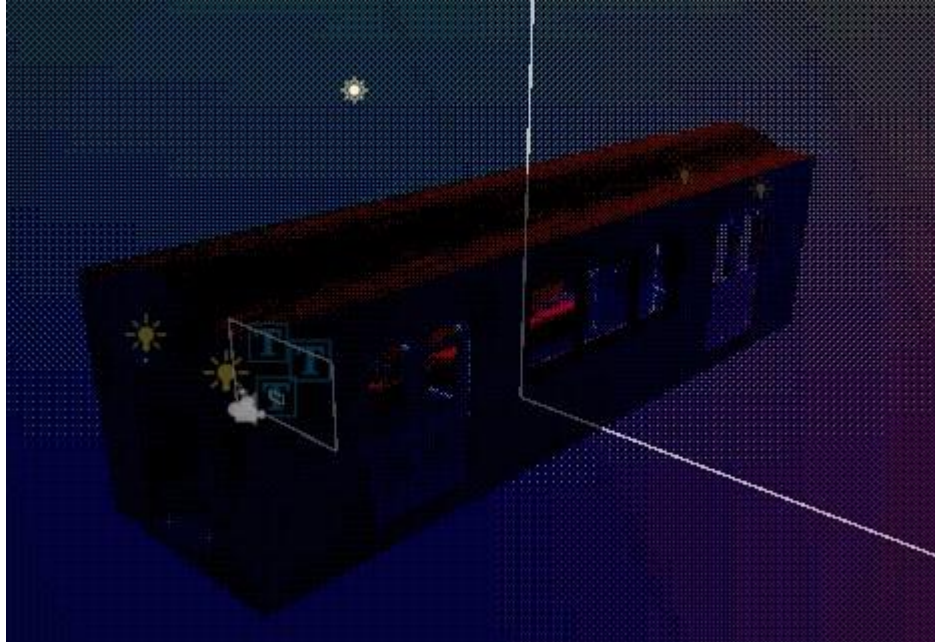


(Figura 3. Escena intermedia entre menú principal y El tren)

InfiniteTrain.cs

Implementa el efecto de tren infinito en la escena Mapa0. El script monitorea la posición del jugador a lo largo del eje de avance del tren y, conforme el jugador se desplaza,

instanciando nuevos vagones delante de él y destruye los que quedan demasiado atrás, manteniendo siempre una cantidad constante de vagones visibles en escena. Este enfoque evita tener una geometría de tren de longitud fija y mantiene el rendimiento estable independientemente del tiempo que el jugador permanezca en el Tren. El script referencia el prefab del vagón para su instanciación dinámica.



(Figura 4. Tren infinito en escena)

StopTrain.cs

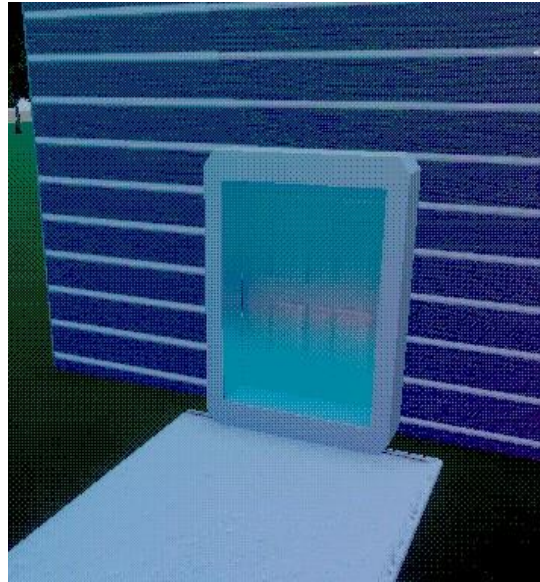
Detecta la pulsación de la tecla 9 y coordina la detención del tren. Al recibir el input, desactiva el componente InfiniteTrain para detener el movimiento y la instanciación de vagones, lanza la animación de frenado del tren, desactiva el AudioSource del motor y detiene la animación de la luz dinámica del interior del vagón. Una vez que las puertas están abiertas, habilita la zona de colisión que activa el script Teleport cuando el jugador la cruza. Centraliza la sincronización entre los sistemas de animación, audio, iluminación y lógica de progresión.

Teleport.cs

Gestiona la transición del jugador desde el Tren hacia el nivel correspondiente. Cuando el jugador cruza el volumen de colisión que StopTrain habilita al abrir las puertas, Teleport llama al SceneManager para cargar la escena del siguiente mundo. El script lleva un registro interno de qué nivel debe cargarse a continuación según el progreso del jugador. La transición es directa: no hay fundido a negro ni pantalla de carga visible; la nueva escena reemplaza a la anterior de forma inmediata.

Portal.cs

Es el equivalente inverso de Teleport: se encarga de regresar al jugador al Tren una vez que completa un nivel. El script está asociado a un objeto de portal que LevelManager activa cuando se alcanzan los diez fragmentos de estrella. Cuando el jugador entra en contacto con el portal, Portal.cs llama al SceneManager para cargar nuevamente la escena Mapa0. De esta forma, el ciclo de juego cierra: el jugador parte del Tren, completa el nivel y regresa al Tren para continuar al siguiente mundo.



(Figura 5. Portal en escena)

LevelManager.cs

Actúa como controlador de estado del nivel actual. Mantiene un contador de fragmentos de estrella recolectados y lo compara contra el objetivo de diez unidades. Cada vez que un fragmento es recogido, el objeto correspondiente notifica a LevelManager, que incrementa el contador y actualiza el elemento de interfaz X/10 visible en pantalla. Cuando el contador llega a diez, LevelManager activa el GameObject del portal, habilitando así el objeto gestionado por Portal.cs para que el jugador pueda regresar al Tren. Este script es el punto central de coordinación entre la recolección de objetos, la interfaz y la condición de completar el nivel.

SaveSystem.cs

Implementa el sistema de guardado y carga de partida. Almacena el progreso del jugador de forma persistente utilizando los mecanismos de serialización de datos de Unity, registrando el nivel alcanzado y los fragmentos recolectados hasta el momento. Este sistema es el que alimenta la opción Cargar del menú principal, permitiendo al

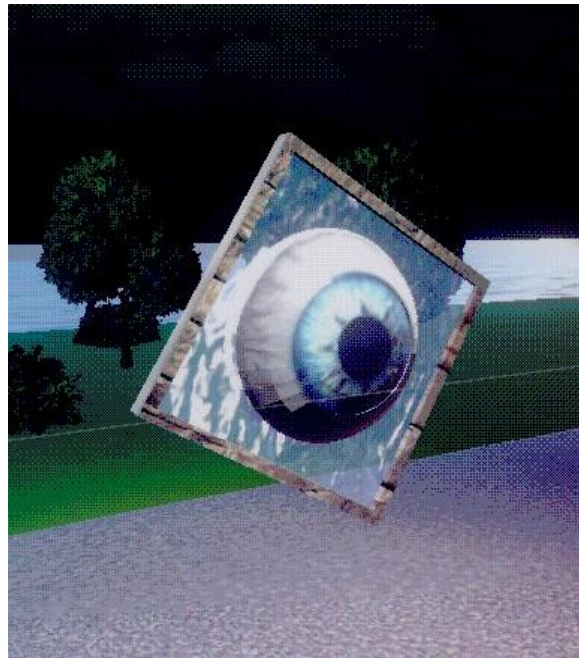
jugador retomar una partida anterior. El guardado se realiza de forma automática al completar cada nivel y regresar al Tren.

MirarJugador.cs

Script de comportamiento aplicado a las entidades NPC del juego. En cada frame, calcula la dirección entre el objeto donde está adjunto y la posición del jugador y rota el objeto para que siempre lo enfrente. El resultado es que las entidades dirigen su mirada hacia el jugador cuando este se encuentra en su radio de influencia, generando el efecto perturbador deseado sin necesidad de un sistema de IA complejo. Es un script ligero sin dependencias externas.

OjoMironGrande.cs

Es una variante extendida de MirarJugador.cs aplicada a entidades específicas que requieren un comportamiento de seguimiento más pronunciado o con parámetros distintos, como velocidad de rotación diferente o un rango de activación mayor. Comparte la misma lógica base de orientación hacia el jugador, pero con valores configurables independientes, lo que permite ajustar el comportamiento de cada tipo de entidad sin modificar el script compartido.



(Figura 6. Entidad con Script OjoMironGrande.cs)

7. Sistema de Renderizado y Shader

7.1 Configuración del Universal Render Pipeline

El proyecto utiliza dos Renderer Assets de URP almacenados en la carpeta Assets/Settings: `pc_renderer`, activo para la build de PC en Windows, y `mobile_renderer`, activo para la build de Meta Quest 2. Ambos comparten la misma configuración base de URP pero difieren en los ajustes de calidad, las sombras en tiempo real y las Renderer Features habilitadas según las capacidades de cada plataforma.

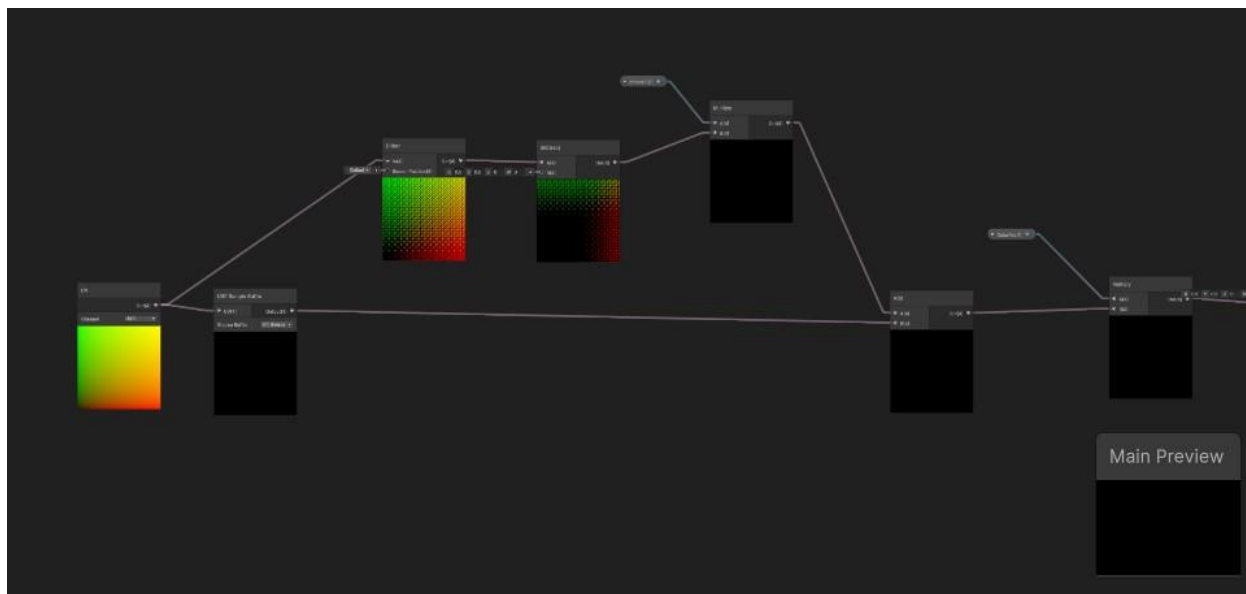
La selección del renderer activo se gestiona desde la Quality Settings del proyecto: el nivel de calidad asignado a la plataforma PC referencia `pc_renderer`, mientras que el nivel asignado a Android referencia `mobile_renderer`. Al generar cada build, Unity aplica automáticamente el renderer correspondiente sin requerir cambios manuales en el proyecto.

7.2 Shader de filtro pixelado

El shader de filtro pixelado es el elemento técnico más visible del proyecto y aplica sobre toda la imagen final que el jugador ve en pantalla. Fue creado con Shader Graph dentro del contexto de URP y está configurado como un Full Screen Shader, lo que significa que opera sobre la textura de salida de la cámara y no sobre geometría individual.

El funcionamiento técnico del shader es el siguiente. Toma como entrada la textura de la cámara renderizada, accesible en URP como `_CameraColorTexture`. Divide las coordenadas UV de la pantalla entre un valor de resolución de cuadrícula que determina el tamaño de cada bloque de píxeles. Aplica la función `Floor` sobre el resultado para redondear las coordenadas al entero inferior más cercano y las multiplica de nuevo por el valor de cuadrícula, generando bloques de color uniforme que simulan una resolución reducida. El parámetro de tamaño de bloque es configurable como propiedad del material, lo que permite ajustar la intensidad del efecto sin recompilar el shader.

El shader está integrado al pipeline mediante una Full Screen Pass Renderer Feature añadida manualmente tanto en `pc_renderer` como en `mobile_renderer`. Esta Renderer Feature inserta el pase del shader al final de la cadena de renderizado, después del postprocesado, de modo que el efecto pixelado se aplica sobre la imagen ya procesada incluyendo bloom, color grading y cualquier otro efecto de volumen activo. Esto garantiza que el filtro sea el último paso visual antes de presentar el frame al jugador.



(Figura 7. Shader Retro para vista del jugador)

8. Iluminación

El proyecto emplea una estrategia de iluminación mixta adaptada a las necesidades visuales y técnicas de cada espacio. La decisión de usar iluminación dinámica o bakeada en cada nivel no es arbitraria: responde directamente al tipo de geometría, la atmósfera deseada y las restricciones de rendimiento de la plataforma Meta Quest 2.

8.1 Mapa1 — Vecindario Tranquilo: iluminación dinámica

El Vecindario Tranquilo utiliza iluminación dinámica en tiempo real. Las fuentes de luz principales son los faroles del vecindario, configurados como Point Lights con sombras en tiempo real habilitadas. Esto permite que las sombras del personaje y de los elementos del entorno respondan al movimiento, reforzando la sensación de presencia del jugador dentro del espacio. El costo computacional de esta configuración es manejable en este nivel porque la geometría es low-poly y el número de luces dinámicas está acotado. Para la build de Meta Quest, las sombras en tiempo real de las luces puntuales se reducen en resolución desde la Quality Settings correspondiente a `mobile_renderer` para mantener el rendimiento dentro de los límites del hardware.



(Figura 8. Iluminación dinámica en Mapa1)

8.2 Mapa2 — Baños: iluminación bakeada

Los Baños utilizan iluminación completamente bakeada mediante el Lightmapper progresivo integrado en Unity. Todas las fuentes de luz del laberinto, luces fluorescentes de tipo Spot y Directional de baja intensidad, están marcadas como Mixed o Baked, y toda la geometría estática del nivel tiene el flag Contribute GI activado. El resultado del bake se almacena como lightmaps en la carpeta Assets/Textures y se aplica automáticamente al cargar la escena.

Esta decisión técnica tiene dos justificaciones. La primera es de atmósfera: la iluminación bakeada permite controlar con precisión los puntos exactos de luz y sombra en cada pasillo del laberinto, creando la oscuridad controlada y los focos tenues que definen la estética del nivel. La segunda es de rendimiento: al eliminar el cálculo de luces dinámicas en un entorno con geometría cerrada y muchos pasillos, se reduce drásticamente el número de draw calls y el costo de renderizado, lo que hace viable ejecutar este nivel en el Meta Quest 2 de forma fluida.



(Figura 8. Iluminación dinámica en Mapa1)

9. Audio

Todo el audio del proyecto fue producido originalmente en FL Studio y exportado en formato .mp3.

Cada escena tiene su propio AudioSource con el clip ambiental correspondiente configurado en modo loop. La escena del Tren tiene un segundo AudioSource dedicado al sonido del motor, que StopTrain.cs desactiva programáticamente al presionar la tecla 9. Esto garantiza que el silencio del motor forme parte de la respuesta coordinada de la mecánica de detención del tren junto con la animación y la luz.

Los parámetros de volumen de música y efectos de sonido son ajustables desde el menú de pausa. Estos valores se gestionan a través de un Audio Mixer de Unity con dos grupos independientes: Music y SFX. Cada AudioSource del proyecto está enrutado al grupo correspondiente, de modo que los controles de volumen del menú afectan globalmente a todos los clips de su categoría sin necesidad de modificar cada fuente de audio individualmente.

10. Sistema de Partículas VFX

Los fragmentos de estrella fugaz utilizan un sistema de partículas de Unity (Particle System) adjunto como componente hijo del GameObject de cada fragmento. El sistema emite partículas luminosas de tamaño pequeño en un radio esférico alrededor del objeto, con un color cálido y una opacidad que varía a lo largo del ciclo de vida de cada partícula mediante una curva de alpha, generando el efecto de brillo pulsante que hace a los fragmentos identificables a distancia.

El sistema de partículas está configurado con emisión continua en loop mientras el objeto está activo en escena. Cuando el jugador recoge el fragmento, el script del objeto llama a LevelManager para registrar la recolección y posteriormente destruye el GameObject completo, incluyendo el Particle System hijo. Esto garantiza que las partículas desaparezcan de forma inmediata junto con el objeto sin dejar rastros visuales en escena.

El número de partículas por sistema está acotado deliberadamente para minimizar el impacto en el rendimiento cuando múltiples fragmentos están visibles simultáneamente en escena. En la build de Meta Quest, el módulo de Renderer del Particle System utiliza el shader de partículas optimizado de URP para móvil, que evita cálculos de iluminación adicionales sobre cada partícula.



(Figura 8. Sistema de partículas de fragmentos de estrella fugaz)

11. Generación de Builds

11.1 Build para PC — Windows

La build de PC se genera desde el Build Settings de Unity seleccionando la plataforma Windows, x86_64. El Renderer Asset activo para esta build es pc_renderer, que habilita sombras en tiempo real, el Full Screen Pass del shader pixelado y los efectos de postprocesado completos por Volume Stack. El resultado es una carpeta con el archivo ejecutable .exe y los datos de la aplicación. El peso total de la build es de aproximadamente 300 MB.

Para ejecutar el juego en PC basta con correr el archivo .exe directamente. No requiere instalación, registro ni conexión a internet. Los requisitos mínimos de hardware se detallan en el Manual de Usuario.

11.2 Build para Meta Quest 2 — APK

La build de Meta Quest se genera cambiando la plataforma activa a Android en el Build Settings y activando el proveedor OpenXR en el XR Plugin Management. El Renderer

Asset activo para esta build es `mobile_renderer`, configurado con Single Pass Instanced Rendering para reducir el costo del renderizado estéreo, Fixed Foveated Rendering en nivel medio para aliviar la carga de shaders en la periferia del campo visual, y resolución de sombras reducida respecto a la build de PC. El Full Screen Pass del shader pixelado también está habilitado en `mobile_renderer`.

El APK resultante pesa aproximadamente 300 MB y debe instalarse en el dispositivo mediante ADB o el Meta Quest Developer Hub con el dispositivo en modo desarrollador activado. Una vez instalado, el juego aparece en la biblioteca de aplicaciones del Quest bajo la categoría de fuentes desconocidas y puede ejecutarse de forma standalone sin conexión a PC.

12. Repositorio y Control de Versiones

El proyecto se encuentra versionado en un repositorio privado de GitHub llamado Proyecto_Final_ARVR. El repositorio contiene el proyecto completo de Unity, incluyendo la carpeta Assets, los archivos de configuración del proyecto y el historial completo de commits que documenta el avance semanal del desarrollo.

El archivo `.gitignore` está configurado específicamente para proyectos de Unity, excluyendo las carpetas Library, Temp, Build, Logs y UserSettings, que se regeneran automáticamente al abrir el proyecto en cualquier equipo y no deben versionarse. Git LFS está habilitado para gestionar los archivos que superan el límite de tamaño estándar de GitHub.

https://github.com/ULTRALEGENDB/Proyecto_Final_ARVR/tree/main

13. Consideraciones Técnicas y Limitaciones Conocidas

El cambio de escena ocurre de forma directa mediante `SceneManager.LoadScene`, lo que puede producir un fotograma de pantalla negra perceptible durante la carga dependiendo del hardware del equipo. Esta decisión es intencional dentro del diseño de la experiencia.

El sistema InfiniteTrain instancia y destruye vagones en tiempo real. En hardware muy limitado, los picos de instanciación pueden causar micro-caídas de fotogramas. Para mitigarlo, el script mantiene un pool pequeño de vagones activos y evita instanciar más de uno por frame.

La escena MapaPruebas no debe incluirse en el Build Settings al generar las builds de producción. Si se incluye accidentalmente, aumenta el peso de la build sin aportar contenido jugable y puede exponer geometría incompleta.

En la build de Meta Quest, si el dispositivo no tiene el modo desarrollador activado, la instalación del APK mediante ADB fallará. Este paso debe completarse desde la aplicación Meta Quest en el teléfono vinculado al dispositivo antes del proceso de instalación.